Town Council Analytics Platform: Complete Technical Documentation

Executive Summary

This document provides comprehensive documentation for the Town Council Analytics Platform, a sophisticated data analytics system designed for Singapore's Town Councils. The platform combines **PySpark** for distributed data processing, **Semantic Kernel** for AI orchestration, **RAG (Retrieval-Augmented Generation)** for intelligent insights, and **KQL databases** for real-time operational analytics. This system enables Town Councils to monitor, analyze, and optimize their estate management operations across all 19 Town Councils in Singapore.

Table of Contents

1. System Overview
2. Architecture Components
3. Data Schema and Design
4. Technical Implementation
5. Analytics Capabilities
6. Deployment Guide
7. Benefits and Impact
8. Future Enhancements
9. Conclusion

1. System Overview

1.1 Purpose and Objectives

The Town Council Analytics Platform addresses the critical needs of Singapore's Town Councils by providing:

Objective	Description
Operational Excellence	Monitor and improve performance across TCMR indicators (Cleanliness, Maintenance, Lift Performance, S&CC Arrears)
Predictive Maintenance	Identify potential failures before they occur through telemetry analysis
Resident Satisfaction	Analyze feedback patterns to improve service delivery
Sustainability Tracking	Monitor energy and water consumption against Green Plan 2030 targets
Data-Driven Decision	Provide actionable insights through AI-powered analysis

1.2 Key Stakeholders

- **Town Council Managers**: Monitor KPIs and operational metrics
- **Maintenance Teams**: Identify and prioritize repair issues
- **Finance Department**: Track S&CC arrears and payment trends
- **Residents**: Indirect beneficiaries of improved services

2. Architecture Components

2.1 PySpark: Distributed Data Processing

What it is: Apache Spark's Python API for large-scale data processing

Why it's used:

- **Scale**: Handles millions of telemetry records across 19 Town Councils
- **Performance**: Distributed processing across multiple nodes
- **Integration**: Seamless with Microsoft Fabric Lakehouse

```
# Example: PySpark operations in the code
lift_enriched_df = lift_df_enhanced \
    .withColumn("fault_hour", hour(col("fault_time"))) \
    .withColumn("fault_month", month(col("fault_time"))) \
    .withColumn("tcmr_lift_rating", when(...))
```

2.2 Semantic Kernel: AI Orchestration

What it is: Microsoft's framework for integrating Large Language Models (LLMs) into applications

Why it's used:

- **Service Management**: Centralized management of AI services (chat, embeddings)
- **Provider Agnostic**: Can switch between OpenAI, Azure OpenAI, or local models
- **Plugin Architecture**: Extensible for custom Town Council operations

```
# Semantic Kernel initialization (though disabled in this version)

self.kernel = sk.Kernel()
self.chat_service = OpenAIChatCompletion(
    service_id="chat_completion",
    ai_model_id=OPENAI_CHAT_MODEL,
    api_key=self.openai_api_key
)
```

Note: In this implementation, we use direct OpenAI calls for embeddings to avoid async complexity, while Semantic Kernel is available for future enhancements.

2.3 RAG (Retrieval-Augmented Generation)

What it is: AI technique that retrieves relevant data to augment prompts before generating responses

Why it's used:

- **Grounding**: Prevents hallucinations by providing actual operational data
- **Contextual Intelligence**: Enables domain-specific responses about estate management
- **Explainability**: Responses reference specific records for transparency

```
# RAG Workflow in the code
async def rag_workflow_async(self, query: str, top_k: int = 5):
    # Step 1: Retrieve relevant context
    context = self.vector_search(query, top_k=top_k)

    # Step 2: Augment prompt with retrieved records
    enhanced_prompt = self.augment_prompt(query, context)

    # Step 3: Generate AI response
    response = await self.generate_response_async(enhanced_prompt)
```

2.4 KQL Database: Real-Time Telemetry

What it is: Kusto Query Language database optimized for time-series and telemetry data

Why it's used:

- **Time-Series Optimization**: Ideal for lift telemetry and utilities consumption
- **Real-Time Ingestion**: Streams data from Tele-Monitoring System (TMS)
- **Fast Query Performance**: Sub-second responses for operational dashboards

3. Data Schema and Design

3.1 Town Council Structure

The system supports all 19 post-GE2025 Town Councils:

```
TOWN_COUNCILS = [
    "Aljunied-Hougang", "Ang Mo Kio", "Bishan-Toa Payoh", "Chua Chu Kang",
    "East Coast", "Holland-Bukit Panjang", "Jalan Besar", "Jalan Kayu",
    "Jurong-Clementi-Bukit Batok", "Marine Parade-Braddell Heights",
    "Marsiling-Yew Tee", "Nee Soon", "Pasir Ris-Changi", "Punggol",
    "Sembawang", "Sengkang", "Tampines", "Tanjong Pagar", "West Coast-Jurong West"
]
```

3.2 HDB Town Mapping

Each Town Council manages specific HDB towns:

```
HDB_TOWNS_BY_COUNCIL = {
    "Ang Mo Kio": ["Ang Mo Kio", "Yio Chu Kang", "Kebun Baru", "Mayflower", ...],
    "Tampines": ["Tampines", "Tampines North", "Tampines East", "Tampines West", ...],
    # ... mappings for all 19 councils
}
```

3.3 Core Datasets

3.3.1 Lift Telemetry (TMS Data)

Field	Type	Description	Business Value
fault_time	Timestamp	Time of fault occurrence	Track fault patterns
town_council	String	Managing council	Accountability
hdb_town	String	Physical location	Geographic analysis
block	String	Block number	Precise location
fault_type	String	Type of fault (door_failure, lift_stuck, etc.)	Root cause analysis
downtime_minutes	Float	Duration of disruption	SLA compliance
severity	Integer	1-5 severity rating	Prioritization

3.3.2 Cleanliness Inspections

Field	Type	Description
inspection_date	Timestamp	When inspection occurred
issue_type	String	stain, litter, graffiti, etc.
severity_weight	Integer	1-5 severity rating
tcmr_rating	String	Green/Amber/Red per TCMR

3.3.3 Maintenance Issues

Field	Type	Description
issue_type	String	obstruction, damaged_plaster, etc.
priority	String	High/Medium/Low
resolution_days	Integer	Days to resolve
sla_met	Boolean	Whether SLA was met

3.3.4 S&CC Arrears

Field	Type	Description
monthly_scc	Integer	Monthly Service & Conservancy Charges
months_in_arrears	Integer	Number of months behind
total_arrears	Float	Total amount owed
repayment_scheme	Boolean	Whether on repayment plan

3.3.5 Utilities Consumption

Field	Type	Description
energy_kwh	Float	Energy consumption
water_m3	Float	Water consumption
location_type	String	hawker_centre, market, residential_common
peak_hour_flag	Boolean	Whether during peak hours

3.3.6 Resident Feedback

Field	Type	Description
category	String	cleanliness, maintenance, lift, etc.
feedback_text	String	Actual feedback content
sentiment	String	positive/neutral/negative
resolution_days	Integer	Days to resolve

3.4 Geospatial Enrichment

All datasets are enriched with latitude/longitude coordinates using OneMap API:

```
def get_coordinates(addresses):
    url = 'https://www.onemap.gov.sg/api/common/elastic/search'
    # Geocodes each HDB town for mapping and spatial analysis
```

This enables:

- **Heat maps** of lift faults across Singapore
- **Proximity analysis** (e.g., lifts within 5km of a location)
- **Geographic clustering** of maintenance issues

4. Technical Implementation

4.1 Data Generation (Synthetic)

The system generates realistic synthetic data for demonstration and testing:

```
def create_lift_telemetry(num_records: int = 5000):
    """Creates realistic lift telemetry with proper Town Council mapping"""
    valid_pairs = []
    for council, towns in HDB_TOWNS_BY_COUNCIL.items():
        for town in towns:
            valid_pairs.append((council, town))
    # Each record gets a valid town-council pair
```

Why synthetic data:

- No real PII required
- Controlled testing scenarios
- Demonstrates full pipeline capabilities

4.2 Data Enrichment Pipeline

Each dataset undergoes enrichment to add business value:

```
# Lift enrichment: TCMR ratings, temporal features
lift_enriched_df = lift_df_enhanced \
    .withColumn("tcmr_lift_rating", when(...)) \
    .withColumn("fault_hour", hour(col("fault_time")))
# S&CC enrichment: risk scoring, intervention flags
scc_enriched_df = scc_df_enhanced \
    .withColumn("arrears_risk_score", when(...)) \
    .withColumn("needs_intervention", when(...))
```

4.3 Embedding Generation

For semantic search, each record gets a vector embedding:

```
@pandas_udf(ArrayType(FloatType()))
def embed_text_batch(texts: pd.Series)-> pd.Series:
    """Generate 1536-dimension embeddings using OpenAI"""
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=clean_texts
    )
    return pd.Series([d.embedding for d in response.data])
```

The operational_context column combines key fields into searchable text:
"LIFT ALERT: Ang Mo Kio | Block BLK-124 | Lift LIFT-107- lift_stuck for 403.7 mins (Severity: 4)"

4.4 Semantic Search Implementation

```
def semantic_search(self, query: str, threshold: float = 0.3):
    # Generate query embedding
    query_vector = self._generate_embedding_sync(query)

    # Compare against all record embeddings
    for row in records:
        similarity = self._cosine_similarity(query_vector, row['metrics_vector'])
        if similarity > threshold:
            results.append(row)
    return sorted(results, key=lambda x: x['similarity_score'], reverse=True)[:top_k]
```

4.5 Heuristic Search (Fallback)

When embeddings aren't available or semantic search yields no results:

```
def heuristic_search(self, query: str):
    if 'lift' in query:
        return self.lift_df.filter(col("severity") >= 3).limit(top_k)
    elif 'clean' in query:
        return self.cleanliness_df.filter(col("issue_type").isin([...]))
    # ... keyword-based routing
```

4.6 Prompt Augmentation

The system builds rich prompts that include actual operational data:

```
enhanced = f"""
RELEVANT OPERATIONAL RECORDS:
1. [LIFT] Ang Mo Kio | Block BLK-124 | lift_stuck | Downtime: 403.7 mins
2. [FEEDBACK] East Coast | Block BLK-099 | cleanliness complaint

USER QUERY: {query}

Provide analysis based on TCMR framework covering:
1) Situation Analysis
2) Observed Patterns
3) Actionable Recommendations
4) Priority Assessment
5) Data-Driven Insights
"""
```

4.7 AI Response Generation

Using OpenAI's GPT-4:

```
async def generate_response_async(self, enhanced_prompt: str):
    response = self.openai_client.chat.completions.create(
        model="gpt-4",
        messages=[
            {"role": "system", "content": "You are a Town Council operations advisor."},
            {"role": "user", "content": enhanced_prompt}
        ],
        temperature=0.3,
```

```
    max_tokens=1000
  )
  return response.choices[0].message.content
```

5. Analytics Capabilities

5.1 TCMR Dashboard

The system generates comprehensive TCMR (Town Council Management Report) summaries:

```
summary = rag.get_town_council_summary("Ang Mo Kio")
# Returns:
# {
#   "lift_stats": {"total_faults": 40, "avg_downtime": 105.96, "high_severity_faults": 22},
#   "cleanliness_stats": {"total_issues": 30, "avg_severity": 1.8},
#   "maintenance_stats": {"total_issues": 44, "resolved": 40, "avg_resolution_days": 6.53},
#   "scc_stats": {"total_households": 42, "arrears_3plus_months": 2, "total_arrears_amount": 850.0}
# }
```

5.2 Semantic Search Examples

Query	What It Finds
"blocks with both lift faults and resident complaints"	Cross-dataset correlations
"households with high S&CC arrears needing intervention"	High-risk cases
"energy consumption peaks in hawker centres"	Sustainability insights

5.3 Geospatial Analysis

With latitude/longitude coordinates, the system enables:

```
# Find lifts within 5km of a location
nearby_lifts = lift_df_with_coords \
  .withColumn("distance_km", distance_udf(lat, lon)) \
  .filter(col("distance_km") < 5)
```

5.4 Cross-Dataset Correlation

The RAG system can identify patterns across datasets:

- Blocks with high lift faults often have negative resident feedback
- S&CC arrears concentrated in specific geographic areas
- Energy consumption patterns correlate with weather data

6. Deployment Guide

6.1 Prerequisites

- Microsoft Fabric workspace with Lakehouse
- OpenAI API key
- Python 3.11+ with PySpark
- OneMap API access for geocoding

6.2 Installation

```
# Install required packages

%pip install openai pandas numpy folium matplotlib
```

6.3 Data Loading from Lakehouse

```
# Load tables from Lakehouse
lift_df = spark.table("lift_telemetry")
cleanliness_df = spark.table("cleanliness_inspections")
maintenance_df = spark.table("maintenance_issues")
scc_df = spark.table("scc_arrears")
utilities_df = spark.table("utilities_consumption")
feedback_df = spark.table("resident_feedback")
```

6.4 RAG System Initialization

```
rag = TownCouncilRAGSystem(
    lift_df=lift_enriched_df,
    cleanliness_df=cleanliness_enriched_df,
    maintenance_df=maintenance_enriched_df,
    scc_df=scc_enriched_df,
    utilities_df=utilities_enriched_df,
    feedback_df=feedback_enriched_df,
    openai_api_key=OPENAI_API_KEY,
    use_kernel=False
)
```

6.5 Sample Queries

```
# Get town summary
summary = rag.get_town_council_summary("Tampines")
# Semantic search
results = rag.semantic_search(
    "blocks with high lift downtime and negative feedback",
    top_k=5
)

# Full RAG workflow
response = await rag.rag_workflow_async(
    "What are the top priorities for improving lift performance in Tampines?"
)
```

7. Benefits and Impact

7.1 Operational Benefits

Area	Benefit
Lift Performance	Reduced downtime through predictive maintenance
Cleanliness	Targeted inspections based on issue patterns

Area	Benefit
Maintenance	Improved SLA compliance through prioritization
S&CC Arrears	Early intervention for at-risk households
Utilities	Energy optimization aligned with Green Plan 2030

7.2 Strategic Benefits

- **Data-Driven Decisions**: Evidence-based resource allocation
- **Resident Satisfaction**: Faster response to feedback
- **Compliance**: Automated TCMR reporting
- **Scalability**: Pattern works for all 19 Town Councils

8. Future Enhancements

1. **Real-time Streaming**: Integrate with Azure Event Hubs for live telemetry
2. **Predictive Models**: ML models for fault prediction
3. **Power BI Dashboards**: Visual dashboards for executives
4. **Mobile App**: Field inspection app for maintenance teams
5. **Weather Integration**: Correlate utilities with weather data

9. Conclusion

The Town Council Analytics Platform represents a modern approach to estate management in Singapore. By combining big data technologies (PySpark), AI orchestration (Semantic Kernel), intelligent retrieval (RAG), and real-time databases (KQL), the system empowers Town Councils to operate more efficiently, respond faster to resident needs, and achieve better outcomes across all TCMR indicators.

The modular architecture ensures that as new data sources become available or AI models improve, the system can easily adapt, making it a future-proof investment for Singapore's public housing management.